

Mood Garden

Technical spec — Phase 1: single flower prototype. Emotion-responsive OpenCV + Unity project.

1. Purpose and scope

Mood Garden is a webcam-driven plant growth toy. A Python process reads facial expression from the webcam and computes two continuous signals — valence (positive/negative expression) and calm (inverse of facial movement variance) — and streams them to a Unity scene over UDP. Unity uses those signals to drive a single flower's growth, bloom, and wilt state in real time.

This phase builds exactly one flower. Do not build a multi-plant pot, additional plant types, a rare-flower unlock system, or menu/UI polish in this phase. Those are documented in Section 8 (Future Phases — Do Not Build Yet) for context only. If a task in this document is ambiguous, resolve it in the direction of less scope, not more.

Reference art

Two reference photos (supplied separately, not embedded in this PDF) anchor the two visual extremes the flower should blend between:

- Bloomed state:** upright magenta/pink celosia plume, fully open, saturated color, stem rigid and vertical.
- Wilted state:** plume browned and drooped over near the top of the stem, desaturated color, stem has lost rigidity and sags.

The flower model does not need to be photorealistic. A stylized low-poly celosia (a central stem with a textured/particle-based plume head) is sufficient. Prioritize a convincing blend between the two states over surface detail.

2. Architecture

Two independent processes on the same machine, communicating over local UDP. Neither process should hard-depend on the other being present — the Unity side should hold its last known state and idle gracefully if no packets arrive.

Component	Responsibility	Tech
Emotion server	Webcam capture, face landmark tracking, emotion classification, signal smoothing, UDP send	Python 3, OpenCV, mediapipe, fer/deepface
Unity client	UDP receive, signal smoothing, drive flower shader/rig parameters, render scene	Unity 2022 LTS or newer, C#, URP

3. Repository layout

This structure already exists (or should be created first if not). Do not nest one process inside the other's folder.

```
mood-garden/  
  ■■■ unity/           # Unity project root
```

```
■■■ python-emotion-server/ # webcam/OpenCV process
■■■ docs/                  # this spec, design notes
■■■ .gitignore
■■■ README.md
```

4. Python emotion server — spec

4.1 Signals to compute

Signal	Range	Meaning	Computation
valence	-1.0 to 1.0	Negative = sad/frowning, positive = smiling	From classifier's happy/sad probabilities, or (happy - sad)
calm	0.0 to 1.0	1.0 = very still/focused, 0.0 = high facial movement	1 - normalized landmark position variance over a sliding time window

4.2 Pipeline

- 1. Capture webcam frame (OpenCV VideoCapture, target 15-30 fps capture; does not need to match send rate).
- 2. Run mediapipe face mesh to get 468 landmarks. If no face detected, do not send stale emotion data — either skip the packet or send an explicit no_face flag (see 4.4).
- 3. Run emotion classifier (fer or deepface) on the face region to get per-class probabilities.
- 4. Compute raw valence from classifier output.
- 5. Maintain a rolling buffer (last 3-5 seconds) of landmark positions; compute variance; derive raw calm.
- 6. Smooth both raw values with an exponential moving average (EMA) before sending — do not send raw per-frame noise. Suggested alpha: 0.15-0.25 (lower = smoother/slower to react).
- 7. Send the smoothed state over UDP at a fixed rate of 10 Hz (every 100ms), independent of capture/inference frame rate.

4.3 Libraries

opencv-python, mediapipe, fer (or deepface if fer proves unreliable), numpy. Pin versions in a requirements.txt at python-emotion-server/requirements.txt.

4.4 UDP message schema

Send newline-free UTF-8 JSON, one object per packet, to 127.0.0.1:5555 (make host/port constants at the top of the script, not hardcoded inline).

```
{
  "valence": 0.62,
  "calm": 0.81,
  "face_detected": true,
  "timestamp": 1234.567
}
```

When `face_detected` is false, valence and calm should hold their last smoothed values rather than snapping to zero — Unity will use `face_detected` to decide whether to keep progressing state or pause.

4.5 Acceptance criteria for this component

- Running the script standalone prints/logs valence, calm, and `face_detected` to console at 10 Hz.
- Smiling broadly for a few seconds visibly moves valence toward +1.0 in the console output; frowning moves it toward -1.0.
- Holding a still, neutral expression for 10+ seconds visibly moves calm toward 1.0; talking or moving the head a lot keeps calm low.
- Covering the webcam or stepping out of frame sets `face_detected` to false within roughly 1 second and does not crash the process.
- No unhandled exceptions on webcam disconnect/reconnect.

5. Unity client — spec

5.1 Receiving state

- A `UdpReceiver` `MonoBehaviour` opens a `UdpClient` on a background thread listening on port 5555 and writes each parsed packet into a thread-safe latest-value buffer (do not touch Unity APIs from the listener thread).
- A separate `EmotionState` singleton/manager reads the latest buffered packet once per `Update()` and exposes smoothed public floats: `CurrentValence` and `CurrentCalm`, each `Mathf.Lerp'd` toward the incoming raw value every frame (not snapped) so visual changes are gradual.
- If no packet has arrived for more than 2 seconds, `EmotionState` should hold its last values rather than reset to zero, and expose an `IsConnected` bool other scripts can check.

5.2 Flower behavior

A single `FlowerController` script on the flower `GameObject` reads `EmotionState.CurrentValence` each frame and drives a normalized 0-1 `bloomAmount` value with its own internal smoothing/inertia (the flower should not visibly track every small fluctuation — it should feel like it is slowly growing or wilting, not twitching).

- **Stem angle:** lerp between upright (`bloomAmount` = 1) and drooped (`bloomAmount` = 0), e.g. rotate a stem bone/pivot between 0 degrees and a drooped angle (roughly 45-60 degrees) — use whatever value looks right against the reference photo.
- **Color:** lerp the plume material's color/tint between the saturated magenta of the bloomed reference photo and a desaturated brown, driven by `bloomAmount`.
- **Scale/fullness (optional if time allows):** slightly scale down or shrink the plume head as `bloomAmount` approaches 0, to read as "withering" rather than just recoloring.
- `bloomAmount` should change slowly over time based on accumulated valence, not jump directly to match `CurrentValence` — e.g. `bloomAmount += CurrentValence * growthRate * Time.deltaTime`, clamped to [0, 1]. Expose `growthRate` as a public/serialized field so it can be tuned in the Inspector without touching code.

The calm signal is received and exposed in this phase but does not need to drive any visual yet — wire it through `EmotionState` so Phase 2 (rare flower unlock) can use it later, but do not build the unlock mechanic now.

5.3 Scene

One scene, one pot, one flower. Camera framing similar to the reference photos (three-quarter view of the pot on a surface). No UI overlay, no menus, no other plants in this phase.

5.4 Acceptance criteria for this component

- With the Python emotion server running, smiling at the webcam for 10-20 seconds visibly moves the flower toward the bloomed reference state; sustained frowning/neutral moves it toward wilted.
- Stopping the Python emotion server does not crash or freeze the Unity scene — the flower holds its current state.
- Restarting the Python emotion server reconnects without needing to restart Unity.
- No visible per-frame jitter/twitching in stem angle or color when expression fluctuates rapidly.

6. Explicit non-goals for this phase

Do not implement any of the following, even if they seem like small additions along the way:

- Additional plants (petunias, vine, basil) from the reference photo — those are Phase 2+.
- The sustained-calm rare flower unlock mechanic.
- Save/load of garden state between sessions.
- Any menu, settings screen, or webcam device picker UI.
- Sound/music.
- Networking beyond localhost (no remote/multiplayer).
- Packaging a standalone build/installer.

7. Suggested build order

- 1. Set up repo structure and .gitignore (Unity + Python templates) if not already done.
- 2. Build the Python emotion server standalone; verify against 4.5 acceptance criteria using console output only, before writing any Unity code.
- 3. Build the Unity UdpReceiver + EmotionState, logging received values to the Console, before touching any visuals.
- 4. Build/import a placeholder flower (even a simple primitive) and wire bloomAmount to a trivial visual (e.g. scale) to confirm the full pipeline works end to end.
- 5. Swap in the real flower model/material and implement the stem-angle and color blending described in 5.2.
- 6. Tune growthRate, EMA alpha values, and lerp speeds by feel against the reference photos.

8. Future phases (context only — do not build yet)

Included so the architecture chosen now does not need to be redone later. Do not act on this section in Phase 1.

- **Phase 2:** Add petunia cluster and trailing vine as additional plant slots in the same pot, each with independent bloom/wilt state.
- **Phase 3:** Sustained-calm streak tracker (using the calm signal already piped through in Phase 1) that unlocks a rare flower type not present in the base pot.
- **Phase 4:** Polish pass — ambient audio, particle effects (falling petals on wilt, pollen/sparkle on bloom), simple UI showing current emotional state.

End of Phase 1 spec. If anything here conflicts with a later instruction from Anshi in the same session, the later instruction wins — but flag the conflict rather than silently overriding scope.